

操作系统实验文档

Lab 8 - 用户进程 User Process

1769fe48

Built by github-actions, at 2025-04-08 13:54:17+08:00

1. 每周实验

1.1 用户进程

1.1.1 实验目的

1. 了解用户地址空间
2. 掌握fork\exec\exit\wait的实现原理

🚩 xv6-lab6 代码分支

<https://github.com/yuk1i/SUSTech-OS-2025/tree/xv6-lab6>

使用命令 `git clone https://github.com/yuk1i/SUSTech-OS-2025 -b xv6-lab6 xv6lab6` 下载 xv6-lab6 代码。

使用 `make run` 运行本次 Lab 的内核，它会启动第一个用户进程 `init`，`init` 会启动 Shell 进程 `sh`。

你会看到 `sh` 进程的用户空间结构。

```
init: starting sh
[INFO 0,2] exec: exec-ed sh, mm structure:
mm 0xffffffffd000fff48:
pgt: 0xffffffffc080d25000
ref: 1
vma: 0xffffffffd010bfd38
[0x00000000ffffe8000, 0x00000000ffff0000), flags: ---U-WR-
[0x0000000000406000, 0x0000000000406000), flags: ---U-WR-
[0x0000000000405000, 0x0000000000406000), flags: ---U-WR-
[0x0000000000404000, 0x0000000000405000), flags: ---U--R-
[0x0000000000402000, 0x0000000000404000), flags: ---UX-R-
=== PageTable at 0xffffffffc080d25000 ===
...
```

注：本次 Lab 的 `struct mm` 初始化代码和上次 Lab 代码中有所变动。

1.1.2 Address Space

我们首先回顾一下，在内核和 CPU 的视角下 U-mode 下的地址空间是如何进行描述的。

对于 CPU 而言，U-mode 下的所有访存，包括取指 (IF)、内存读 (LD)、内存写 (ST) 都是要通过页表进行地址翻译的。而 CPU 是通过 CSR `satp` 作为页表基地址来进行地址翻译的。

所以，内核需要为用户进程设置页表结构。在内核中，我们使用 `struct mm` 管理用户内存，每个 PCB `struct proc` 中均有一个指向 `struct mm` 的指针 `*mm`。

```
// os/vm.h
struct mm {
    spinlock_t lock;

    pagetable_t __kva pgt;
    struct vma* vma;
};
```

每个 `struct mm` 中有一张页表 `pgt`，它就是用户进程使用的 `satp`，我们可以在 `usertrapret` 和 `trampoline` 中观察到这一点：

```
// os/trap.c
void usertrapret() {
    // ...

    // tell trampoline.S the user page table to switch to.
    uint64 satp = MAKE_SATP(KVA_TO_PA(curr_proc()->mm->pgt)); // <--
    uint64 stvec = (TRAMPOLINE + (uservec - trampoline)) & ~0x3;

    // jump to userret in trampoline.S at the top of memory, which
    // switches to the user page table, restores user registers,
    // and switches to user mode with sret.
    uint64 fn = TRAMPOLINE + (userret - trampoline);
    ((void (*)(uint64, uint64, uint64))fn)(TRAPFRAME, satp, stvec);
}

// os/trampoline.S
.globl userret
userret:
    # userret(TRAPFRAME, pagetable, stvec)
    # switch from kernel to user.
    # usertrapret() calls here.
    # a0: TRAPFRAME, in user page table.
    # a1: user page table, for satp.
    # a2: uservec

    # switch to the user page table.
    csrw satp, a1
    sfence.vma zero, zero
```

在内核的视角下，**用户的内存空间是几个连续的虚拟地址区域**。我们将每个连续的区域用 `struct vma` (Virtual Memory Area) 结构体表示，并使用链表将它们串起来。每个 `vma` 中有这一个区域的起始地址和结束地址（要求对齐到页边界），以及该区域的权限。

```
// os/vm.h
struct vma {
    struct mm* owner;
    struct vma* next; // linked list
    uint64 vm_start; // start address (user virtual address)
    uint64 vm_end; // end address (user virtual address)
    uint64 pte_flags; // flags
};
```

注意到 `struct vma` 中有一个 `struct mm*` 指针，这表示每个 VMA 都下属于某个 `struct mm`。

Loading

用户 程序 会指定它在加载时，需要加载哪几个连续的区域（也成为段 Segment）。以下是 `llvm-readelf-19 -a user/build/sh` 的输出，表示 `sh` 程序加载（LOAD）时会需要三个连续的区域：

```
Elf file type is EXEC (Executable file)
Entry point 0x402000
There are 4 program headers, starting at offset 64

Program Headers:
  Type           Offset       VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  ATTRIBUTES     0x00c3dc 0x0000000000000000 0x0000000000000000 0x000061 0x000000 R  0x1
  LOAD           0x001000 0x0000000000040200 0x0000000000040200 0x0011f4 0x0011f4 R E 0x1000
  LOAD           0x003000 0x0000000000040400 0x0000000000040400 0x0000cd 0x0000cd R  0x1000
  LOAD           0x004000 0x0000000000040500 0x0000000000040500 0x000020 0x0007d0 RW 0x1000
```

我们可以在 `os/proc.c` 中的 `exec` 函数中，使用 `mm_print` 打印 `sh` 加载完成后进程的 `struct mm` 结构以及 `pgt` 页表。

```
mm 0xffffffffd000fff88:
  pgt: 0xffffffffc080b14000
  ref: 1
  vma: 0xffffffffd010bfe28
    [0x00000000fffe8000, 0x00000000ffff0000), flags: ---U-WR-
    [0x000000000406000, 0x000000000406000), flags: ---U-WR-
    [0x0000000000405000, 0x0000000000406000), flags: ---U-WR-
    [0x0000000000404000, 0x0000000000405000), flags: ---U--R-
    [0x0000000000402000, 0x0000000000404000), flags: ---UX-R-
=== PageTable at 0xffffffffc080b14000 ===
[0], pte[0xffffffffc080b14000]: 0x0000000000000000 -> 0x00000000080b22000 -----V
[2], pte[0xffffffffc080b22010]: 0x0000000000400000 -> 0x00000000080b23000 -----V
[2], pte[0xffffffffc080b23010]: 0x0000000000402000 -> 0x00000000080b21000 ---UX-RV
[3], pte[0xffffffffc080b23018]: 0x0000000000403000 -> 0x00000000080b20000 ---UX-RV
[4], pte[0xffffffffc080b23020]: 0x0000000000404000 -> 0x00000000080b1f000 ---U--RV
[5], pte[0xffffffffc080b23028]: 0x0000000000405000 -> 0x00000000080b1e000 ---U-WRV
[3], pte[0xffffffffc080b14018]: 0x00000000c0000000 -> 0x00000000080b18000 -----V
[1ff], pte[0xffffffffc080b18ff8]: 0x00000000ffe00000 -> 0x00000000080b19000 -----V
[1e8], pte[0xffffffffc080b19f40]: 0x00000000fffe8000 -> 0x00000000080b1d000 ---U-WRV
[1e9], pte[0xffffffffc080b19f48]: 0x00000000fffe9000 -> 0x00000000080b1c000 ---U-WRV
[1ea], pte[0xffffffffc080b19f50]: 0x00000000fffea000 -> 0x00000000080b1b000 ---U-WRV
[1eb], pte[0xffffffffc080b19f58]: 0x00000000fffeb000 -> 0x00000000080b1a000 ---U-WRV
[1ec], pte[0xffffffffc080b19f60]: 0x00000000fffec000 -> 0x00000000080b24000 ---U-WRV
[1ed], pte[0xffffffffc080b19f68]: 0x00000000fffed000 -> 0x00000000080b25000 ---U-WRV
[1ee], pte[0xffffffffc080b19f70]: 0x00000000fffee000 -> 0x00000000080b26000 ---U-WRV
[1ef], pte[0xffffffffc080b19f78]: 0x00000000fffef000 -> 0x00000000080b27000 ---U-WRV
[ff], pte[0xffffffffc080b147f8]: 0x00000003fc000000 -> 0x00000000080b15000 -----V
[1ff], pte[0xffffffffc080b15ff8]: 0x00000003fffe0000 -> 0x00000000080b16000 -----V
[1fe], pte[0xffffffffc080b16ff0]: 0x00000003fffffe000 -> 0x00000000080b17000 DA---WRV
[1ff], pte[0xffffffffc080b16ff8]: 0x00000003fffff000 -> 0x00000000080b20b000 -A--X-RV
=== END ===
```

我们可以发现，在 vma 链表中，有三个区域是和 ELF 中的 LOAD 段相同的，它们分别是 `.text`，`.rodata`，`.data/.bss` 段。

```
[0x0000000000402000, 0x0000000000404000), flags: ---UX-R-
[0x0000000000404000, 0x0000000000405000), flags: ---U--R-
[0x0000000000405000, 0x0000000000406000), flags: ---U-WR-
```

另一个以 `0xfffe` 开始的地址则是进程的栈区。

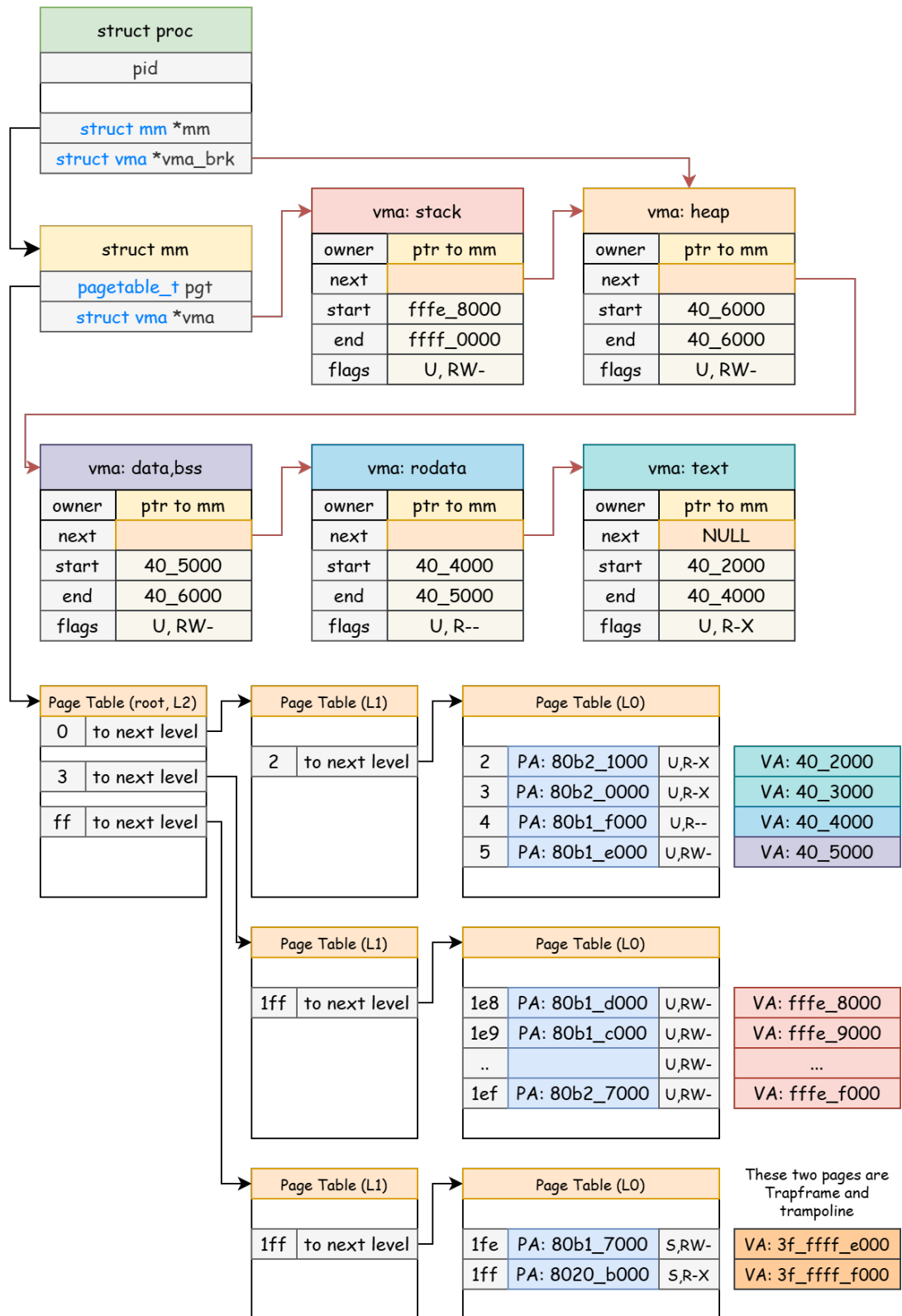
```
[0x00000000fffe8000, 0x00000000ffff0000), flags: ---U-WR-
```

此外，还有一个大小为0的区域，它跟随在所有 LOAD 段后面，这是进程的堆区 (Heap，在远古的操作系统上我们称之为 Program Break)。进程需要使用 `sbrk` 系统调用来扩展或收缩堆区。

```
[0x0000000000406000, 0x0000000000406000), flags: ---U-WR-
```

概览图

该图展示了内核中用于管理 `sh` 进程的用户地址空间的数据结构。



1.1.3 fork

fork 系统调用是操作系统中用于创建一个新进程的函数。当一个进程调用 fork 时，它会创建一个与父进程几乎完全相同的新进程（称为子进程）。子进程会复制父进程的地址空间、所有寄存器值。唯一不同的是，fork 调用返回的值：

- 父进程：fork 返回子进程的 PID（进程ID）。
- 子进程：fork 返回 0。

xv6 中的 fork 实现位于 os/proc.c 中的 fork 函数。以下为简化版本：

```
int fork() {
    int ret;
    struct proc *np = allocproc(); // child process
    struct proc *p = curr_proc(); // parent process

    // create a new struct mm for child process
    np->mm = mm_create(np->trapframe);

    // Copy user memory from parent to child.
    mm_copy(p->mm, np->mm);

    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);

    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;
    np->parent        = p;

    // add the child process to scheduler
    np->state          = RUNNABLE;
    add_task(np);

    return np->pid; // return value for the parent process
}
```

fork 的调用者即为父进程，其中用 allocproc 申请的新 PCB 即为子进程。我们通过修改 np 的 trapframe 来实现两者拥有不同的返回值。注意到我们并没有改 p 的 trapframe，这是因为 syscall 函数在分发系统调用时，会为我们将 fork 函数的返回值写入 trapframe 中的 a0，我们只需让 fork 返回子进程的 PID 即可。

mm_copy（位于 os/vm.c）函数最终实现了对所有 vma 的复制：实际上就是在新的 mm 下面创建新的 struct vma，赋值 vma 中的属性，调用 mm_mappages 映射该 vma，最后复制实际的内存数据。

⚠ Trapframe 和 Trampoline

在概览图中我们可以发现，用户页表中包含 Trapframe 和 Trampoline，而 vma 链表中并不包含这两个页面。这样的设计是刻意的而非bug。

考虑 vma（用户的 Virtual Memory Area）的 **生命周期** (Lifecycle)，exec 系统调用会删除所有现有的用户内存映射并替换为新的，但是 PCB (即 struct proc) 对象是不需要销毁重建的，Trapframe 似乎也不需要重新分配物理页面。

所以，Trapframe 和 Trampoline 的生命周期实际上与该进程一致，而不是与任何一个 vma 条目一致。

在实现上，我们在系统初始化时，即 proc_init 中申请 Trapframe 页面，在 create_mm 中映射 Trampoline 和 Trapframe。

1.1.4 exec

exec 系统调用用于执行一个新的程序，并用新的程序替换当前进程的内存空间。与 fork 不同，exec 不会创建一个新的进程，而是用一个新程序替换当前进程的代码、数据和堆栈。

调用 exec 时，当前进程的地址空间会被新的程序的代码和数据替换掉。原先进程的代码、数据、堆栈都会被清空，并且加载新的程序。随后，当前进程的执行流将跳转到新程序的入口点，继续执行新的程序代码。

```
int exec(char *name, char *args[]) {
    struct user_app *app = get_elf(name);

    struct proc *p = curr_proc();

    // execve does NOT preserve memory mappings:
    // free VMAs including program_brk, and ustack
    // load_user_elf() will create a new mm for the new process and free the old one
    // , if page allocations all succeed.
    // Otherwise, we will return to the old process.
    // However, keep the phys page of trapframe, because it belongs to struct proc.
    load_user_elf(app, p, args);

    // syscall() will overwrite trapframe->a0 to the return value.
    return p->trapframe->a0;
}
```

load_user_elf 函数相较于上次 Lab 有所变动。我们需要注意到，**申请物理页面是有可能失败的**。在系统没有足够的内存时，exec 函数执行失败，我们需要回到原来的进程中继续执行，并且要释放掉分配到一半的内存（即不要漏内存）。

所以，我们创建一个新的 `struct mm`，并向其中加载 ELF 中的段和进程栈等。只有不再需要进行内存分配后 (`mm_mappages`)，我们才将 `p->mm` 给清空并覆盖掉。

```
// os/loader.c, Simplified version.
int load_user_elf(struct user_app *app, struct proc *p, char *args[]) {

    // create a new mm for the process, including trapframe and trampoline mappings
    struct mm *new_mm = mm_create(p->trapframe);

    Elf64_Ehdr *ehdr      = (Elf64_Ehdr *)app->elf_address;
    for (int i = 0; i < ehdr->e_phnum; i++) {
        struct vma *vma = mm_create_vma(new_mm);
        // Load Segment from phdr.
        if (mm_mappages(vma) < 0)    // if page allocation fails, jump to bad.
            goto bad;
    }
    // setup brk: zero
    mm_mappages(vma_brk);
    // setup stack
    mm_mappages(vma_ustack);

    // from here, we are done with all page allocation
    // (including pagetable allocation during mapping the trampoline and trapframe).

    // free the old mm.
    if (p->mm)
        mm_free(p->mm);

    // we can modify p's fields because we will return to the new exec-ed process.
    p->mm      = new_mm;
    // setup trapframe
    p->trapframe->epc = ehdr->e_entry;

    return 0;

    // otherwise, page allocations fails. we will return to the old process.
bad:
    warnf("load (%s) failed: %d", app->name, ret);
    mm_free(new_mm);
    return ret;
}
```

1.1.5 生命周期

1.1.6 exit

`exit` 系统调用用于终止当前进程，并返回一个退出状态给操作系统。`exit` 系统调用永远不会返回。调用 `exit` 后，进程的一些资源 **不会被操作系统立刻回收**。并且，`exit` 不会使其立刻从父进程的视野中消失。它仍然存在于“僵尸进程”状态，直到父进程通过 `wait` 系统调用获取到子进程的退出状态并回收该进程。

以下是简化版本的 `exit`。注意到在这里面我们并没有回收用户资源：到目前的 Lab 为止，我们只介绍过一种用户资源，即用户内存。

在 `exit` 中，我们只将自己的状态设为 `ZOMBIE`，并把退出代码保存到 `p->exit_code` 中。然后，我们使用 `wakeup` 唤醒自己的父进程。

```
// Exit the current process.
void exit(int code) {
    int wakeinit = 0;
    struct proc *p = curr_proc();

    acquire(&wait_lock);
    // reparent

    // wakeup wait-ing parent.
    wakeup(p->parent);

    acquire(&p->lock);
    p->exit_code = code;
    p->state     = ZOMBIE;
    release(&wait_lock);

    sched();
    panic_never_reach();
}
```

⚠️ wakeup

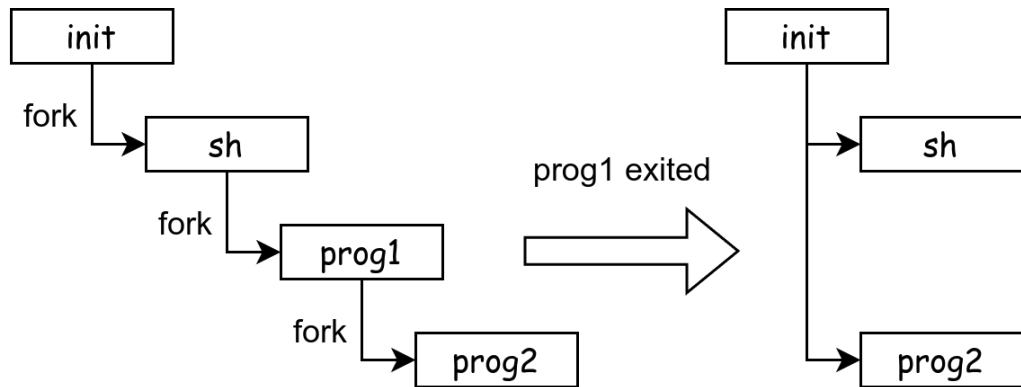
我们会在后续的同步 Lab 具体讲解锁和同步，以及上述代码中的 `wait_lock` 是什么。

在这节 Lab，我们只需要知道父进程在 `wait` 找不到处于 `ZOMBIE` 的子进程时会陷入睡眠。而子进程 `exit` 会把自己设为 `ZOMBIE`，这打破了父进程陷入 **Sleeping** 的条件，所以我们唤醒父进程使其不再睡眠。

reparent

"reparent"是指当一个进程的父进程终止时，操作系统会将该进程的父进程更改为 `init` 进程（PID 为 1 的进程）。这是一个系统级的管理机制，用于确保在父进程终止时，子进程仍然有一个父进程来进行资源回收、进程管理等必要操作。

reparent 机制主要是为了避免“孤儿进程”问题，并确保系统资源得到适当回收。在 `xv6` 中，`init` 就像是一个“孤儿院”，负责回收子进程们创建的孙子进程。



```
// user/src/init.c

for (;;) {
    // this call to wait() returns if the shell exits,
    // or if a parentless process exits.
    wpid = wait(-1, NULL);
    if (wpid == pid) {
        // the shell exited; restart it.
        printf("init: sh exited, restarting...\n");
        break;
    } else {
        // it was a parentless process; do nothing.
        printf("init: wait a parentless process %d\n", wpid);
    }
}
```

1.1.7 wait

`wait` 系统调用是操作系统中用于进程控制的一个重要函数。它的主要作用是允许一个父进程等待其子进程的状态变化（在 `xv6` 中只有结束这一种情况），并且将子进程的退出代码返回给父进程。

在 `xv6` 中，`wait` 的原型更像是 Linux 中的 `waitpid(2)`。传入的 `pid` 可以为一个负值，表示 `wait` 任何一个子进程，否则表示 `wait` 特定的子进程。

在 `xv6` 中，为了实现简单，表示父进程和子进程之间的关系使用的是子进程 PCB 中的 `parent` 指针，而并没有一个反向的 `children` 列表用于遍历。所以，我们遍历所有的进程，试图找到一个 `child` 的 `parent` 指针为当前进程 (`curr_proc()`)。

当我们能找到这样的进程，并且它是 `ZOMBIE` 状态时，我们即可对它进行资源回收，以及从 `wait` 系统调用中返回。

```
int wait(int pid, int *code) {
    struct proc *child;
    int havekids;
    struct proc *p = curr_proc();

    acquire(&wait_lock);

    for (;;) {
```

```

// Scan through table looking for exited children.
havekids = 0;
for (int i = 0; i < NPROC; i++) {
    child = pool[i];
    if (child == p)
        continue;

    acquire(&child->lock);
    if (child->parent == p) {
        havekids = 1;
        if (child->state == ZOMBIE && (pid <= 0 || child->pid == pid)) { // condition for
matching a child process
            int cpid = child->pid;
            // Found one.
            if (code)
                *code = child->exit_code;
            freeproc(child);
            release(&child->lock);
            release(&wait_lock);
            return cpid;
        }
    }
    release(&child->lock);
}

// No waiting if we don't have any children.
if (!havekids || p->killed) {
    release(&wait_lock);
    return -ECHILD;
}

debugf("pid %d sleeps for wait", p->pid);
// Wait for a child to exit.
sleep(p, &wait_lock); // DOC: wait-sleep
}
}

```

`freeproc` 函数会释放 `child` 的 PCB（将其标记为 UNUSED），并回收用户内存（`mm_free`）。

1.1.8 Sleep 机制

想象一下，一个进程正在使用 `read` 系统调用从 `stdin`（即控制台）上读取数据，但是此时用户还没有输入任何内容。操作系统有两种途径处理这种情况：

1. 暂时让该进程停止，等到控制台上有了数据了再返回。
2. 一直轮询控制台输入，有输入了就立刻从 `read` 中返回。

显然的，第二种情况对 CPU 这种宝贵的资源而言是不高效的。控制台等 Input/Output 操作是没有响应时间上的上限。与其让 CPU 空转，不如先将 CPU 出让给其他进程来执行，等到控制台有了数据了再唤醒原来的进程。这即是我们为什么需要让进程陷入睡眠：因为我们缺乏某些条件继续执行下去；与其浪费 CPU，不如等满足条件时再被唤醒。

对于 Console 这样的例子而言，缺乏的条件就是用户没有输入内容；那么如何在 Console 上有输入时被唤醒呢：回想我们之前的课程，我们应该让 Console 在有数据时向 CPU 发出中断。

sleep & wakeup

`sleep` 和 `wakeup` 函数的实现如下，我们使用一个 `void*` 指针表示一个任意数据，它表示当前进程因为什么原因陷入 `SLEEPING`。

`sleep` 会将 `curr_proc()->state` 设为 `SLEEPING`，并调用 `sched()` 函数切换到 scheduler。注意到 scheduler 只会挑选 `RUNNABLE` 的进程来执行，所以当前进程再也不会被 scheduler 调度到了，即它再也不会继续执行了。

`wakeup` 则遍历所有进程，如果它的睡眠原因 `sleep_chan` 和传入的 `chan` 相同，则将其重新设为 `RUNNABLE`，并使它能被 scheduler 继续执行到。

```
void sleep(void *chan, spinlock_t *lk) {
    struct proc *p = curr_proc();

    acquire(&p->lock); // DOC: sleeplock1
    // Go to sleep.
    p->sleep_chan = chan;
    p->state = SLEEPING;

    sched();
    // p get waking up, Tidy up.
    p->sleep_chan = 0;

    release(&p->lock);
}

void wakeup(void *chan) {
    for (int i = 0; i < NPROC; i++) {
        struct proc *p = pool[i];
        acquire(&p->lock);
        if (p->state == SLEEPING && p->sleep_chan == chan) {
            p->state = RUNNABLE;
            add_task(p);
        }
        release(&p->lock);
    }
}
```

lock

注意到 `sleep` 函数会传入一把自旋锁 `spinlock_t *lk`，在这节课上我们暂且不需要理解为什么要这样设计。

Console

以控制台为例，`read` 系统调用会最终来到 `os/console.c: user_console_read` 函数。如果内核中的缓冲区 `cons.buffer` 为空 (`cons.r == cons.w`)，则令当前进程在 `&cons` 上陷入睡眠。

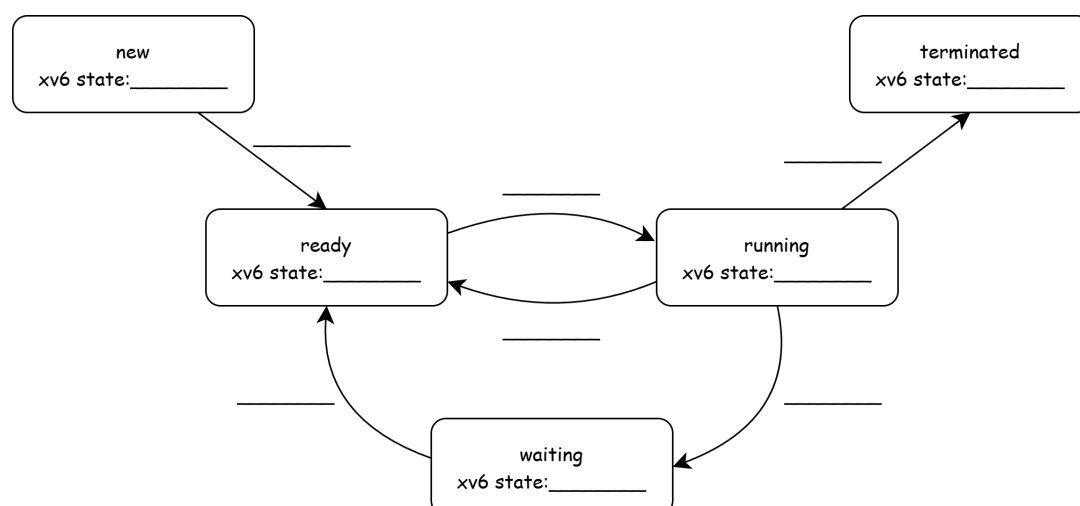
当控制台设备上有可以读取的数据时，Console 会通过 PLIC 向 CPU 发起中断，在 `trap.c`: `plic_handle()` 中，我们通过 `irq` 判断中断是否来自于 Console (UART)。如果是，则分发到 `uart_intr` 中处理。`uart_intr` 会从 UART 设备中读取一个字节 `uartgetc`，并调用 `consintr`，在我们收到一个 `\n` 时，我们唤醒在 `&cons` 上等待的进程。

```
static void consintr(int c) {
    if (c == '\n' || c == C('D') || cons.e - cons.r == INPUT_BUF_SIZE) {
        // wake up consleread() if a whole line (or end-of-file)
        // has arrived.
        cons.w = cons.e;
        wakeup(&cons);
    }
}
```

1.1.9 课堂报告

1. 参照 xv6 内核，从用户程序调用 `fork` 系统调用开始，依次列出之后操作系统会执行哪些重要函数，直到子进程从 `fork` 中返回 0 为止。这些"重要函数"为管理以下子系统（或功能）的函数：PCB (Process Control Block), User Address Space (only `vm.c`), Trap, CPU Scheduler, Context Switch。你可以假设函数调用永远返回成功，不需要考虑错误处理的路径。
2. 填写下表，将理论课上讲解的进程状态映射到 xv6 中 `enum procstate` 的取值 (`struct proc, state`)，并写出在哪些 **事件发生** 时状态会发生转变。

注：左上角 new 状态对应 `procstate` 中的 `USED`。



3. 在不同的进程中，它们的 Trampoline 是同一张物理页面吗？它们的 Trapframe 是同一张物理页面吗？

在 `sh >>` 下执行命令 `test_arg 123 asd`，`sh` 会 `fork&exec` 启动 `test_arg` 程序。内核也会打印 `test_arg` 的页表。观察它的页表和 `sh` 的页表中的 Trapframe 和 Trampoline。

4. 在使用 `exec` 系统调用时，我们会传入进程的参数，即 `int exec(char *path, char *argv[])`。我们会在 `main` 函数中收到 `argv` 数组：`int main(int argc, char *argv[])`，其中 `argv` 是一个 `char*`（字符串指针）的数组，它的最后一项是 `NULL`，`argc` 表示这个数组中有多少个字符串指针。

我们知道 `exec` 会删除所有内存映射。那么，旧进程进行 `exec` 系统调用时传入的 `argv` 是怎么传入到新进程的？

Hint: 在 `make run` 后，`applist` 中有 `test_arg`。你可以在 `sh >>` 中测试它。

Debugger 教程：使用 `make debug` 运行带调试器的 QEMU，使用 `gdb-multiarch` 连接 QEMU，先使用两次 `continue` 使内核运行到 `sh >>` 提示符。

在 `gdb` 中按 `Ctrl+C` 中断，使用 `add-symbol-file user/build/test_arg` 添加用户程序的调试符号，执行 `b main` 在用户程序的 `main` 函数打断点。

执行 `continue` 继续运行内核。在内核命令行输入 `test_arg asdf asfkjls asf`（后面内容随便敲）然后回车执行，你应该能在 `gdb` 中收到位于用户进程的断点。

使用 `stack -l 20` 观察此时栈上的内容。（注意 `0xffff0000` 往后的地址是未映射的），右侧 `a1` 表示 `a1` 寄存器的值为该地址 `0x00000000ffffeff0`。

```
(qemu) gef> stack -l 20
0x00000000ffffeff70|+0x0000: 0x00000000ffffeffd0 → 0x0000000000006466 → 0x0000000000006466 ← $sp
0x00000000ffffeff78|+0x0008: 0x000000000040234c → 0x02813c8303813b83 → 0x02813c8303813b83
0x00000000ffffeff80|+0x0010: 0x00000000ffffefff0 → 0x6772615f74736574 → 0x6772615f74736574 ← $fp,
$a1
0x00000000ffffeff88|+0x0018: 0x00000000ffffeffe8 → 0x0000343135343131 → 0x0000343135343131
0x00000000ffffeff90|+0x0020: 0x00000000ffffeffe0 → 0x0030313839313931 → 0x0030313839313931
0x00000000ffffeff98|+0x0028: 0x00000000ffffeffd8 → 0x0066647361667361 → 0x0066647361667361
0x00000000ffffeffa0|+0x0030: 0x00000000ffffeffc8 → 0x7361667361667361 → 0x7361667361667361
0x00000000ffffeffa8|+0x0038: 0x00000000ffffeffb8 → 0x7769756168667361 → 0x7769756168667361
0x00000000ffffeffb0|+0x0040: 0x0000000000000000 → 0x0000000000000000
0x00000000ffffeffb8|+0x0048: 0x7769756168667361 → 0x7769756168667361
0x00000000ffffeffc0|+0x0050: 0x000000000b666265 → 0x000000000b666265
0x00000000ffffeffc8|+0x0058: 0x7361667361667361 → 0x7361667361667361
0x00000000ffffeffd0|+0x0060: 0x0000000000006466 → 0x0000000000006466
0x00000000ffffeffd8|+0x0068: 0x0066647361667361 → 0x0066647361667361
0x00000000ffffeffe0|+0x0070: 0x0030313839313931 → 0x0030313839313931
0x00000000ffffeffe8|+0x0078: 0x0000343135343131 → 0x0000343135343131
0x00000000ffffefff0|+0x0080: 0x6772615f74736574 → 0x6772615f74736574
0x00000000ffffefff8|+0x0088: 0x0000000000000000 → 0x0000000000000000
```

注意到 `0x7769756168667361` 这样的数字像是 ASCII 码，使用 `x/60s $sp` 打印栈上像是字符串的东西。

注：在计算机底层中，“字符串”是以 `0x00` 结尾的一串字节。

1.1.10 推荐阅读

1. Trampoline 和 Trapframe 的物理页面是哪里来的？

参照上述我们打印的 `sh` 进程的用户页表，注意其中最后两条映射的物理地址：

```
[ff], pte[0xffffffc080b147f8]: 0x0000003fc0000000 -> 0x0000000080b15000 -----V
[1ff], pte[0xffffffc080b15ff8]: 0x0000003fffe00000 -> 0x0000000080b16000 -----V
[1fe], pte[0xffffffc080b16ff0]: 0x0000003fffffe000 -> 0x0000000080b17000 DA---WRV
[1ff], pte[0xffffffc080b16ff8]: 0x0000003ffffff000 -> 0x0000000080b20b000 -A--X-RV
```

参照 "Week 6 - 内核页表 Kernel Paging" 中的 "xv6 内核内存布局" 一章，找出这两个物理地址分别位于哪个物理地址区域。对照源代码 `trampoline.S` 和 linker script `os/kernel.ld` 验证你的答案。

2. 回顾上周的课堂报告问题：Trapframe 和 Trampoline 是两个页面，这两个页面应该允许 U-mode 访问吗？

请你亲自做实验验证。

对于 Trampoline，修改 `kvm.c` 中 `kvmmake` 函数，调用 `kvmmap` 映射 trampoline 处，在权限中 OR 上 `PTE_U`。

对于 Trapframe，修改 `proc.c` 中 `allocproc` 中，调用 `mm_mappage_at` 函数处，在权限中 OR 上 `PTE_U`。

使用 `make debug` 以及 `gdb-multiarch` 挂载调试器。使用 `b kernel_trap_entry` 和 `b *0x3fffffff000` 在内核 Trap 入口处和 `uservec` 处打断点，使用 `print $scause` 手动查看 Trap 相关的 CSR。